In [22]:
```
# Experiment:
# Classification Models - Logistic Regression, KNN,
# Naive Bayes and ROC Analysis
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix, classification_report,
roc_curve, roc_auc_score
)
# 1. Load Dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="Target")
print("Dataset Shape:", X.shape)
print("Target Classes:", data.target_names)
# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.20, random_state=42, stratify=y
)
# 3. Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# 4. Create Models
models = {
"Logistic Regression": LogisticRegression(max_iter=5000),
"KNN": KNeighborsClassifier(n_neighbors=5),
"Naive Bayes": GaussianNB()
}


results = {}
roc_data = {}

# 5. Train and Evaluate Models
for name, model in models.items():

    model.fit(X_train_scaled, y_train)

    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1]

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
```

```
    auc = roc_auc_score(y_test, y_prob)

    results[name] = [accuracy, precision, recall, f1, auc]

    fpr, tpr, thresholds = roc_curve(y_test, y_prob)
    roc_data[name] = (fpr, tpr, auc)

    print("\n-------------------------------------")
    print(name)
    print("-------------------------------------")
    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall   :", round(recall, 4))
    print("F1 Score :", round(f1, 4))
    print("AUC      :", round(auc, 4))

    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

# 6. Comparison Table
results_df = pd.DataFrame(
    results,
    index=["Accuracy", "Precision", "Recall", "F1 Score", "AUC"]
).T

print("\nModel Comparison:")
print(results_df)

# 7. ROC Curve
plt.figure(figsize=(9, 6))

for name, (fpr, tpr, auc) in roc_data.items():
    plt.plot(fpr, tpr, label=f"{name} (AUC = {auc:.3f})")

plt.plot([0, 1], [0, 1], linestyle="--",
         label="Random Classifier")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()

plt.grid()
plt.show()
```

```
Dataset Shape: (569, 30)
Target Classes: ['malignant' 'benign']

--------------------------------------
Logistic Regression
--------------------------------------
Accuracy : 0.9825
Precision: 0.9861
Recall   : 0.9861
F1 Score : 0.9861
AUC      : 0.9954

Confusion Matrix:
[[41  1]
 [ 1 71]]

Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.98      0.98        42
           1       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114


--------------------------------------
KNN
--------------------------------------
Accuracy : 0.9561
Precision: 0.9589
Recall   : 0.9722
F1 Score : 0.9655
AUC      : 0.9788

Confusion Matrix:
[[39  3]
 [ 2 70]]

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.93      0.94        42
           1       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114


--------------------------------------
Naive Bayes
--------------------------------------
Accuracy : 0.9298
Precision: 0.9444
```

```
Recall   : 0.9444
F1 Score : 0.9444
AUC      : 0.9868

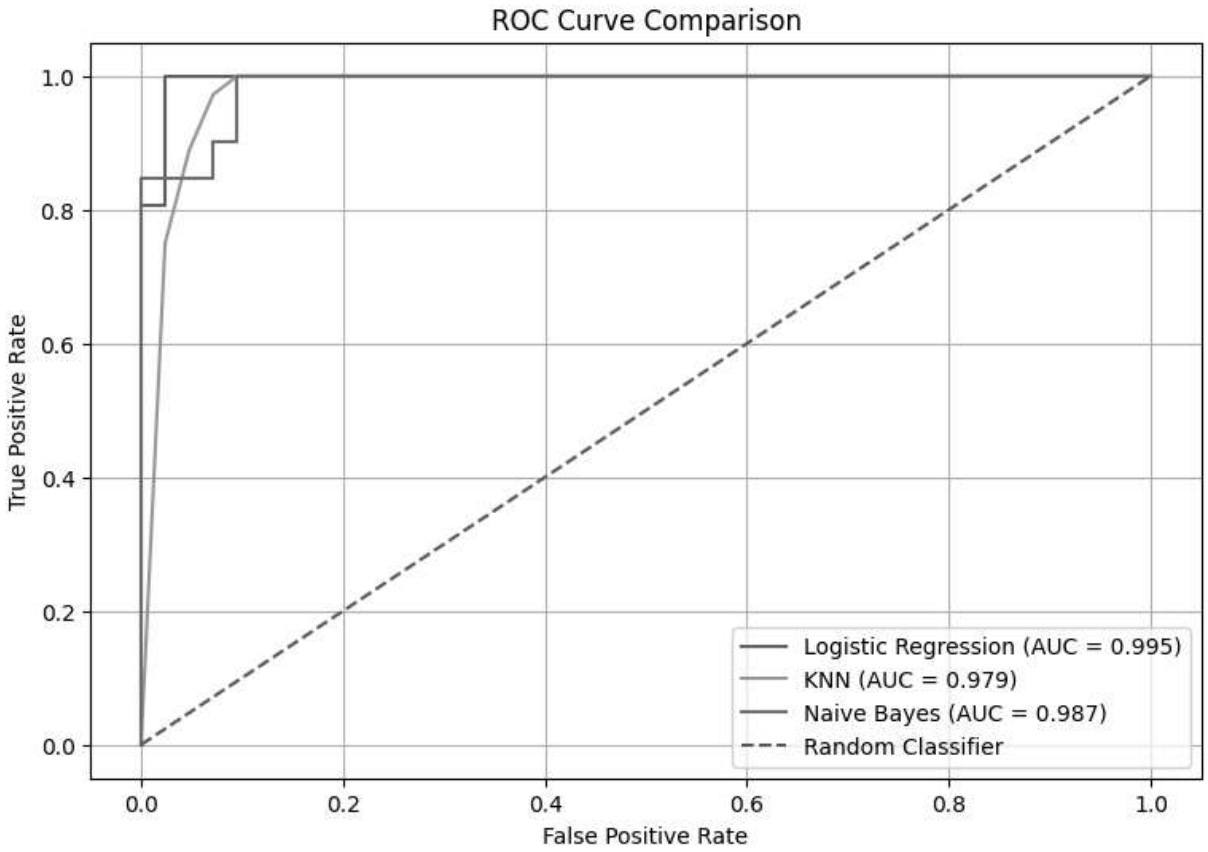
Confusion Matrix:
[[38  4]
 [ 4 68]]

Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.90      0.90        42
           1       0.94      0.94      0.94        72

    accuracy                           0.93       114
   macro avg       0.92      0.92      0.92       114
weighted avg       0.93      0.93      0.93       114


Model Comparison:
                     Accuracy  Precision    Recall  F1 Score       AUC
Logistic Regression  0.982456   0.986111  0.986111  0.986111  0.995370
KNN                  0.956140   0.958904  0.972222  0.965517  0.978836
Naive Bayes          0.929825   0.944444  0.944444  0.944444  0.986772
```



In [19]:
```
!jupyter nbconvert --to html /content/smld_7_new.ipynb
```

```
[NbConvertApp] Converting notebook /content/smld_7_new.ipynb to html
[NbConvertApp] Writing 294942 bytes to /content/smld_7_new.html
```